



The Token Economy

How enterprise AI architecture impacts
cost and utility at scale

Table of contents

The maturing economics of enterprise AI

01

Tokens as AI currency

02

Enterprise AI architecture and token efficiency

03

Context: Retrieving the precise data

3.1

Continual learning: Getting smarter and more efficient with every execution

3.3

Models: Right-sizing intelligence

3.2

Harness: Orchestrating context at scale

3.4

The more AI understands work, the fewer tokens it needs to complete it

04

01

The maturing economics of enterprise AI

AI adoption is accelerating across the enterprise and quickly becoming a core area of technology spend. What began with copilots and chat assistants is now expanding into a much heavier class of workloads, coding agents, productivity coworkers, and workflow automation, all that drive token demand. [Deloitte's](#) 2025 Tech Value Survey found that more than half of respondents now allocate between 21% and 50% of their digital initiative budgets to AI on average, a sign that AI has moved from novelty to meaningful spend.

That budget pressure is rising just as enterprise AI is becoming more agentic. The shift from lightweight chat interactions to long-running agentic workflows dramatically changes the economics of these systems. Modern AI systems continuously retrieve context, call tools, reason across steps, generate intermediate outputs, and pass work between models and agents. Each of those operations consumes tokens, the fundamental unit of compute and cost in AI systems, meaning that as AI workflows become more complex token consumption rises alongside them.

The use of frontier reasoning models compounds the problem further. Frontier models are designed to deliberate, branch, self-verify, and reason over longer horizons before producing output. That additional reasoning improves capability, but because these models are designed to reason over longer horizons, they are often priced at a higher cost per token while also consuming more tokens per task.

Engineering offers an early warning of what happens when AI usage scales faster than economic discipline. In some teams, heavy AI consumption has even become a status signal – a sign that developers know how to push coding agents into ever more ambitious workflows. The emerging discourse around “token-maxing” reflects that shift, with [reports](#) and [anecdotes](#) describing coding-tool budgets being exhausted far earlier than expected and monthly bills reaching startling levels. These examples show how quickly AI demand can scale. They also expose a broader problem: many organizations are still optimizing for maximum AI usage rather than economically efficient AI usage.

That is why the next phase of enterprise AI adoption will not be defined simply by who deploys the most advanced models most aggressively. It will be defined by who can generate the most useful work per token. This is an architectural challenge that depends on enterprise AI architecture to efficiently and accurately retrieve context, route work across models, learn from prior execution, and manage state across long-running tasks. In other words, token efficiency is not just a model-pricing issue; it is also an enterprise AI architecture issue.

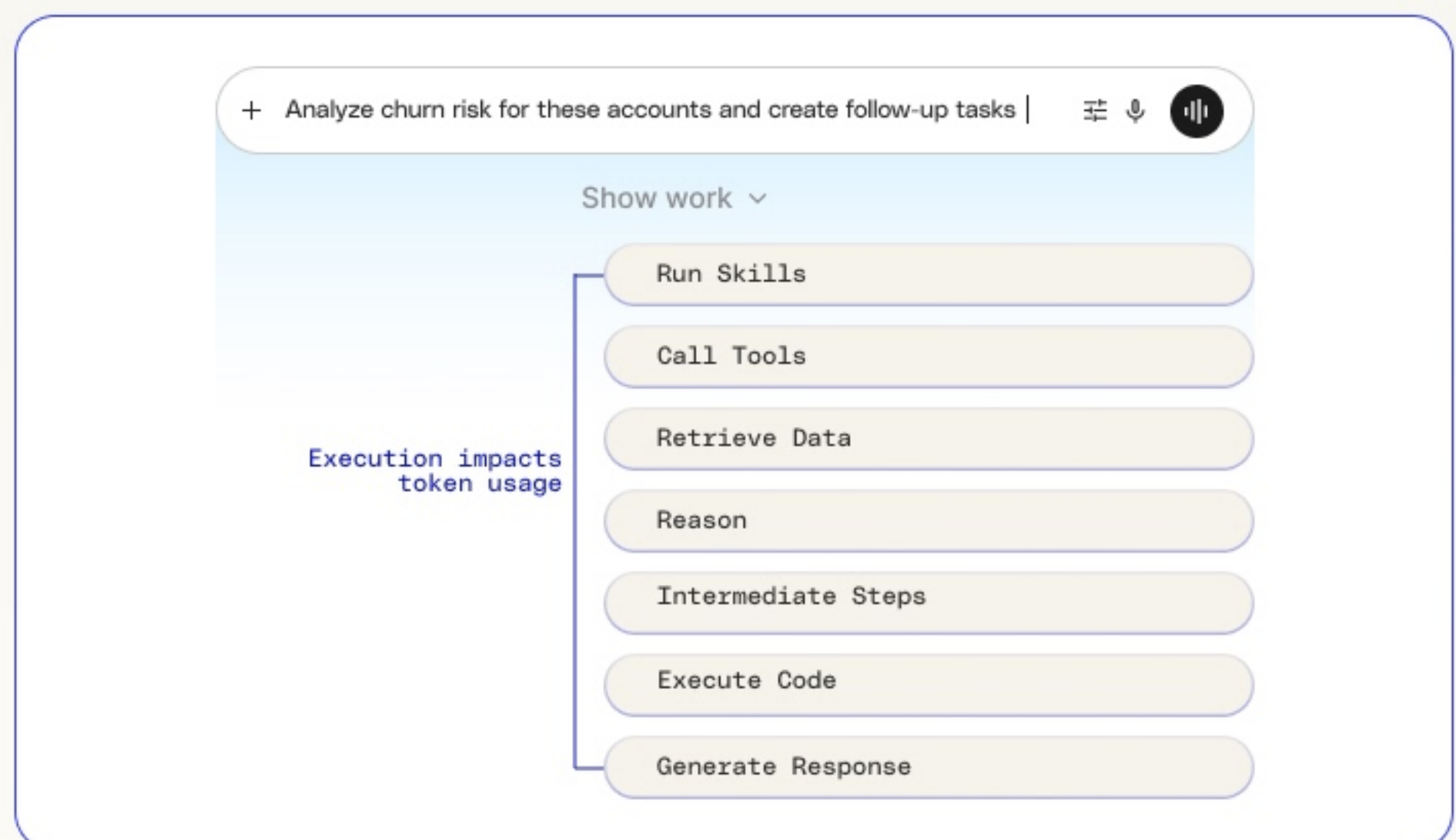
02

Tokens as AI currency

Early in the generative AI cycle, token usage was relatively easy to reason about because most interactions were simple prompt-response exchanges.

That framing breaks down in enterprise AI systems as the majority of tokens flowing through are no longer user-typed text. They are system prompts, tool schemas, retrieved documents, intermediate steps, and reasoning loops generated as the system works through a task.

Consider a routine enterprise request: "Analyze churn risk for these accounts and create follow-up tasks." This execution requires search calls to retrieve account context, skills to understand business analysis, reading of data by sub-agents, and structured output generation as an artifact. Each step injects tokens beyond the original prompt and the reasoning traces generated between steps further contribute to that count. An eleven word prompt can expand into thousands of tokens per execution.



Enterprise AI architecture dictates the sequence and number of execution steps, including reasoning loops, all which impact token usage.

That is why token economics are no longer just a model-pricing issue. The real driver is execution design. If a system repeatedly overfetches context, invokes unnecessary tools, forces a frontier model to do routine retrieval work, or carries bloated state across every step of a workflow, token consumption compounds. If it retrieves precisely, routes intelligently, and manages context with discipline, token consumption falls without necessarily sacrificing quality.

The result is a cost surface expanding across token volume per task, model tier, and context size simultaneously. Understanding where that cost accumulates and where it can be reduced without sacrificing quality is becoming a foundational capability of enterprise AI platforms.

03

Enterprise AI architecture and token efficiency

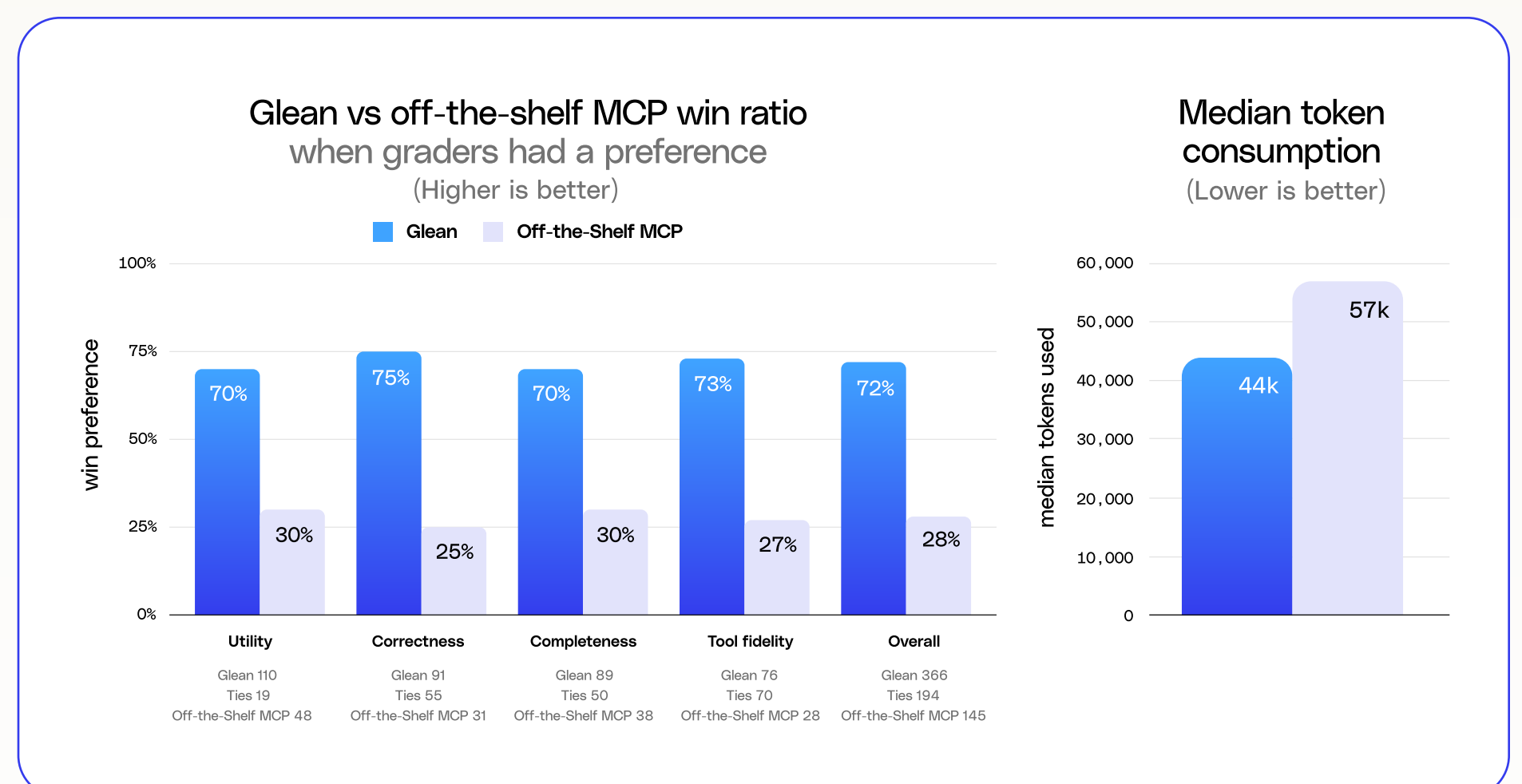
Context: Retrieving the precise data for the task

Enterprise AI starts with context, because before a system can resolve a support ticket, summarize an account, analyze pipeline risk, or triage an IT incident, it needs to understand the underlying enterprise data connected to the task. The quality of that context layer determines both the usefulness of the result and the amount of work the system needs to do to get there.

This matters because LLMs do not inherently know what information matters. They process information as tokens within a context window where every token is mathematically compared and weighted against other tokens through an attention mechanism that becomes increasingly expensive as context grows.

The challenge is not simply fitting more information into a larger context window but rather retrieving the right information with precision so that the model reasons over signal instead of noise. Without well-designed context systems, larger context windows often make reasoning worse rather than better. Models become distracted by irrelevant information, retrieve conflicting signals, and spend additional tokens reconciling ambiguity.

Glean's own [benchmark](#) testing illustrates the point. Glean measured the role context quality plays in token efficiency by comparing Glean's remote MCP server against off-the-shelf MCP servers in Claude Cowork. Glean was preferred approximately 2.5x as often, while off-the-shelf tools consumed roughly 30% more tokens on average. When segmented by outcome, the gap widened further, with off-the-shelf MCP tools requiring nearly double the tokens to achieve winning results, consuming approximately 83k tokens compared to Glean's 43k. In other words, weaker context quality forced the system to spend more tokens searching, retrying, and reconciling before it got to a relevant answer.



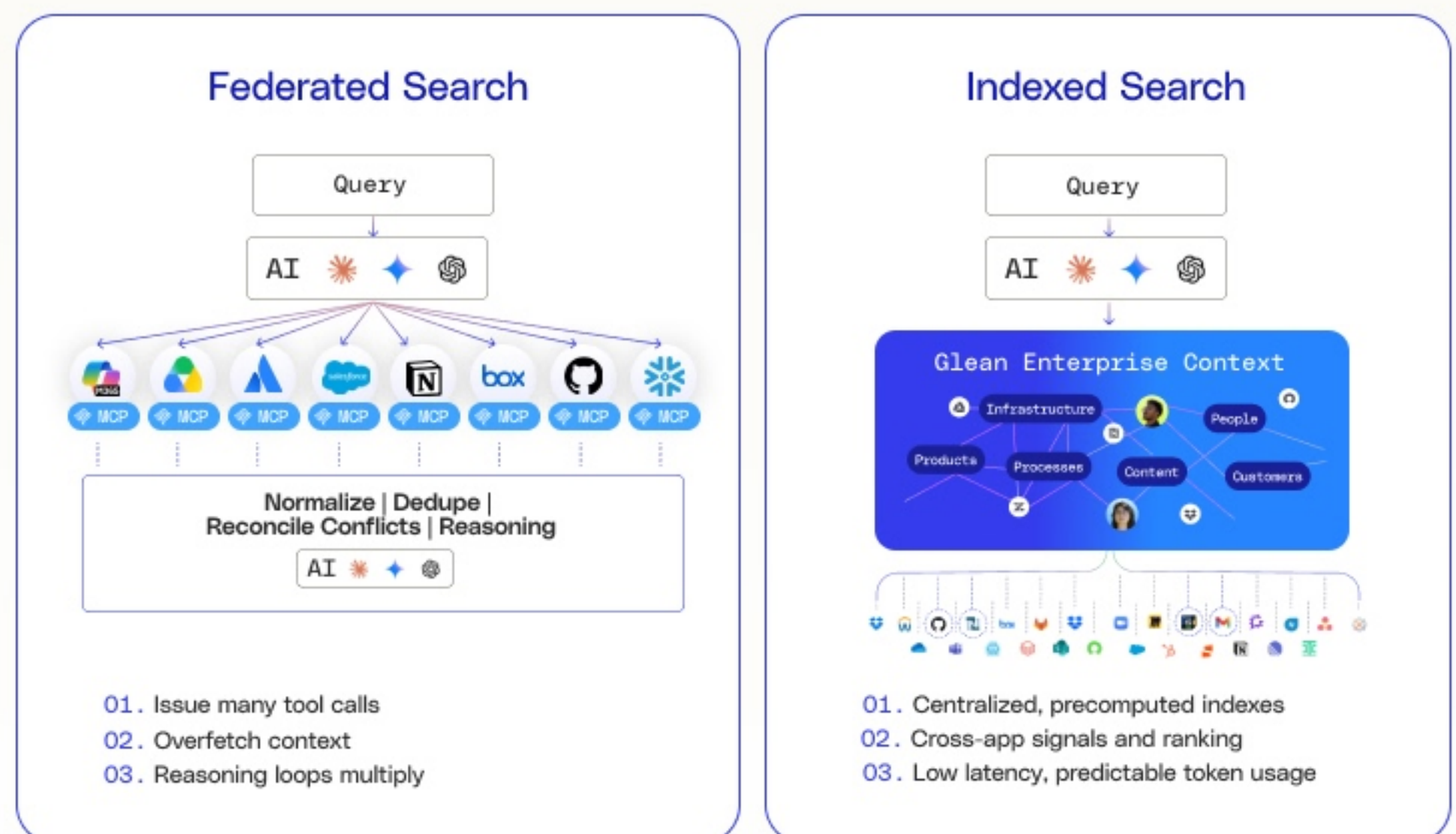
The performance and token efficiency of Glean's remote MCP server versus off-the-shelf-MCP servers on a benchmark of ~175 queries.



The average token usage of Glean versus off-the-shelf MCP per correct answer.

Context design matters when it comes to token efficiency. Glean is built on centralized indexes across more than 100 enterprise applications, including Microsoft, Atlassian, Google, and Salesforce, combined with enterprise knowledge graphs that map cross-application relationships. When a task spans multiple systems, the enterprise graph allows the LLM to traverse those relationships directly rather than inferring them from raw data, producing context that is more precise and more compact.

Off-the-shelf MCP tools work very differently. They may lack semantic or lexical indexes, rely heavily on user-specific signals that reflect only a fraction of the relevant enterprise context. The data they return arrives in heterogeneous shapes and sizes that the LLM must normalize itself, with no reliable basis for weighting one source against another. The burden of disambiguation falls on the LLM at query time rather than on precomputed, centralized indexes that understand the relationships between enterprise data and that matters for token efficiency.



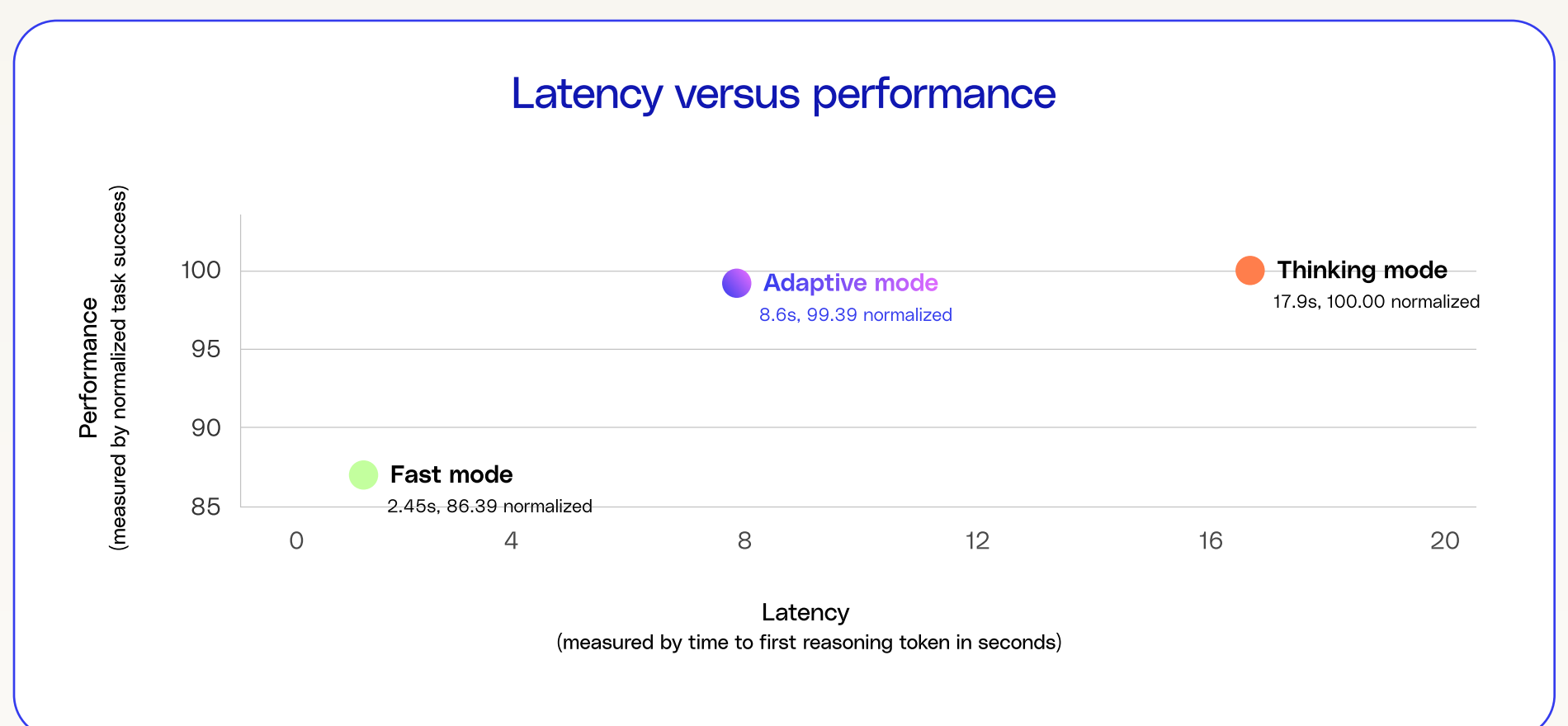
Indexed search queries a unified, pre-processed context, returning results with minimal reasoning. Federated search queries multiple live sources in real time, requiring additional reasoning to normalize results, resolve conflicts, and deduplicate before a response can be returned.

Models: Right-sizing intelligence for the task

Frontier LLMs are valuable for complex reasoning and response generation, but enterprise AI architectures that route every step of an agentic workflow through them are paying frontier costs for tasks that do not require frontier intelligence.

This is why we're seeing the rise of intelligent routing, routing that can determine the right model and reasoning level for the task, alongside the advent of smaller, open-source models that can offload some of the work of frontier models at a fraction of the cost.

At Glean, we introduced [Waldo](#), a specialized small model focused on agentic search, to deliver approximately 50% lower latency and approximately 25% fewer tokens with no quality regression.

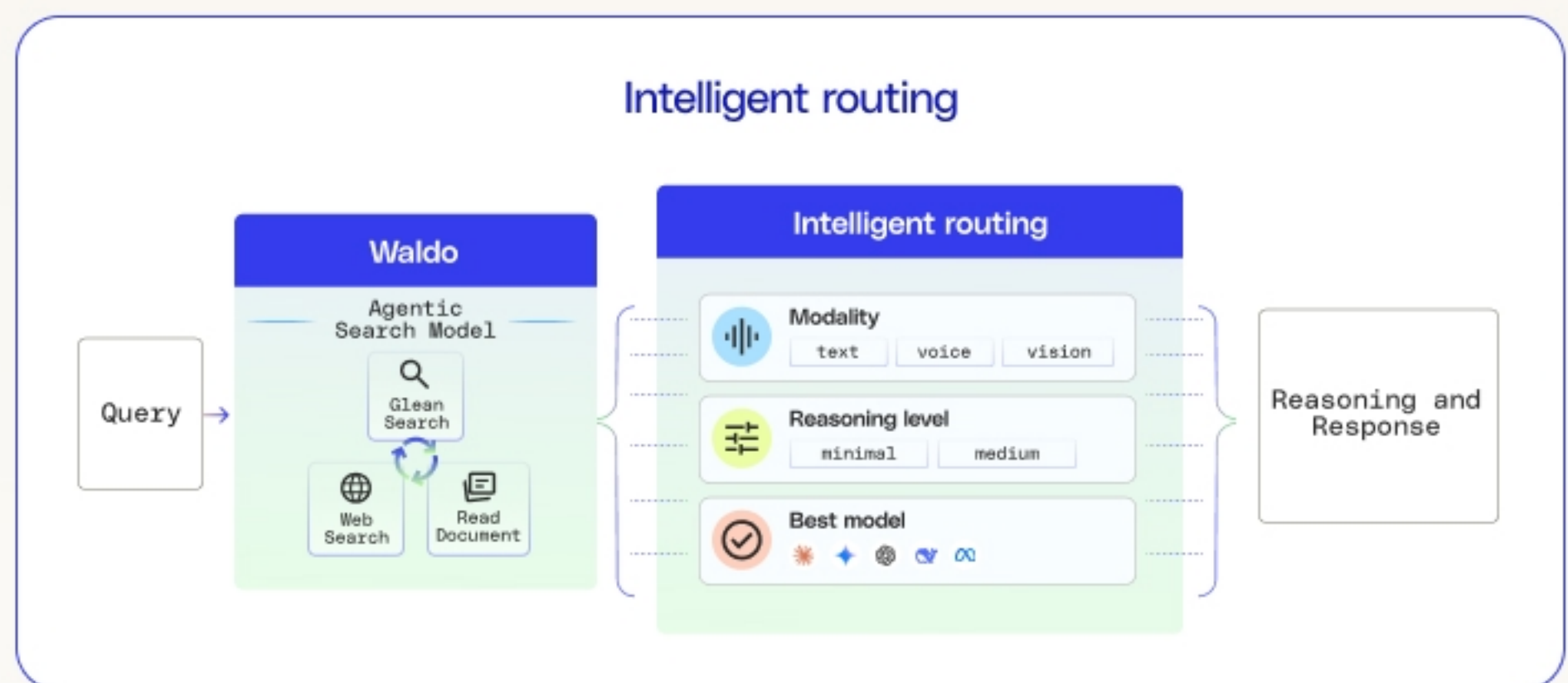


The performance of adaptive mode, which uses Glean Waldo to dynamically determine the reasoning level for each task, compared to fast mode and thinking mode.

Enterprise workloads usually begin with search, regardless of how complex the final deliverable may be. The context-gathering phase is largely operational: requiring pattern matching over query intent, tool selection, and gathering enough evidence to hand off to a frontier LLM for synthesis. Bundling that function into a frontier model meant paying frontier prices for search planning and adding latency on every turn of the loop. Glean's response was to isolate the search planning function into a small, purpose-trained model.

Waldo changes that economics. It is post-trained specifically to run the iterative search phase that precedes synthesis, and it works in concert with frontier models rather than replacing them. It also enables adaptive reasoning. Because Waldo runs first, its own execution becomes a signal for downstream complexity: how many tools were invoked, how many documents were returned, whether results were sparse or rich, and whether the task looks routine or genuinely difficult. Reasoning level becomes a dynamic decision informed by evidence gathered during execution.

Token efficiency at the model layer therefore requires flexibility: the ability to mix frontier and open-source models, route tasks by complexity, and reserve expensive reasoning for the steps that actually need it. The point is not to use the cheapest model possible. Rather, it is to use the right intelligence at the right moment.



Glean intelligently routes across open-source and proprietary models based on reasoning level, modality, and performance evaluations.

Continual learning: Getting smarter and more efficient with every execution

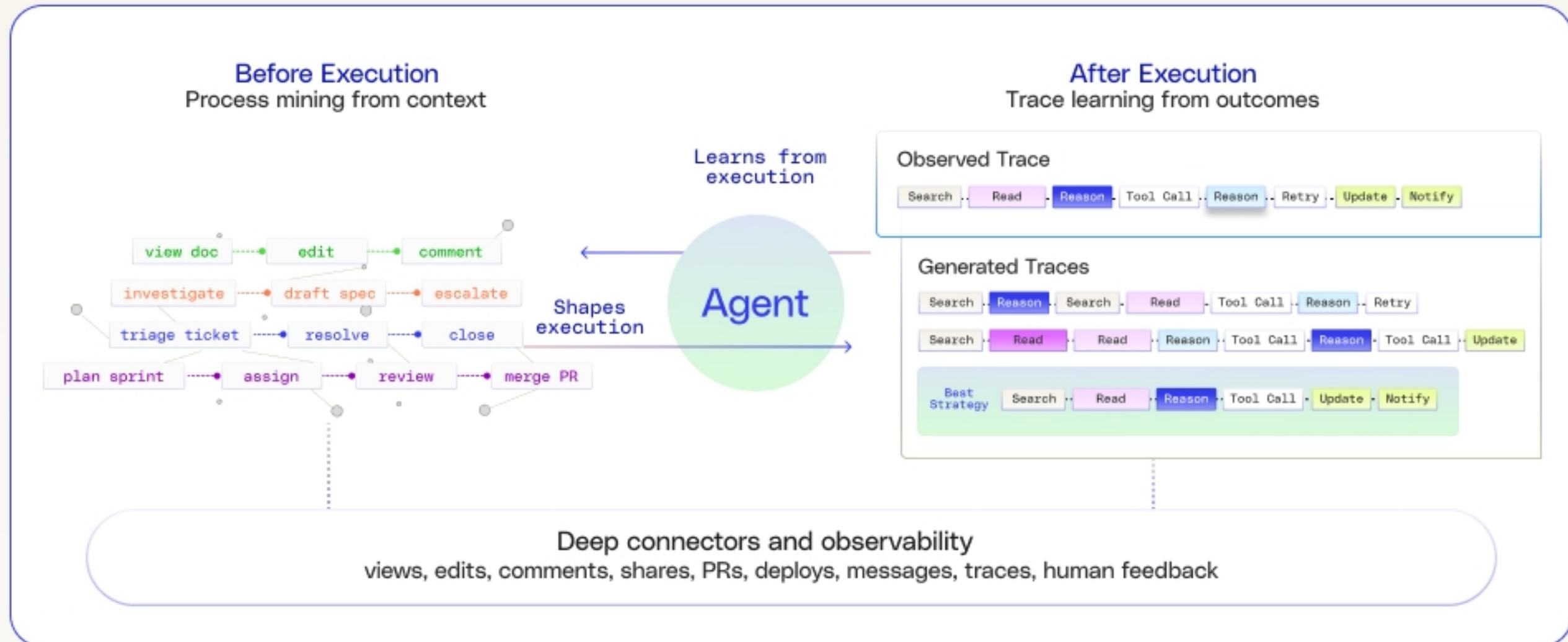
Continual learning is often discussed as a model problem. In enterprise AI, it is at least as much a context and memory problem.

Every time an LLM completes a task, it leaves behind an execution trace: the tool calls it made, the queries it issued, the documents it retrieved, the branches it explored, and the path it took to reach a result. A platform designed for continual learning does not discard that trace. It treats it as evidence. Over time, those traces reveal which routes are direct, which steps are redundant, and which patterns tend to succeed on recurring task types.

That matters economically because repeated work should get cheaper. A mature enterprise AI system should not rediscover the same solution path from scratch every time a familiar task appears. It should learn the contours of the work, recognize recurring structures, and approach similar tasks with a stronger prior understanding of how they are usually completed.

This kind of learning extends beyond direct interactions between a user and an AI system. Enterprises already generate a huge volume of latent process knowledge in the form of tickets, approvals, handoffs, edits, escalations, and repeated workflow patterns across systems and teams. Most AI systems ignore those signals. A stronger architecture turns them into usable context: evidence of how work actually gets done over time. When an agent encounters a similar task later, it can follow the relevant process pattern instead of exploring the solution space from scratch.

That is what makes continual learning strategically important. Each completed task contributes evidence that makes the next similar task more efficient to complete, narrowing the gap between the open-ended exploration that characterizes early agentic deployments and the targeted, experience-informed execution of mature enterprise AI systems. Over time, the system becomes both smarter and cheaper to run.



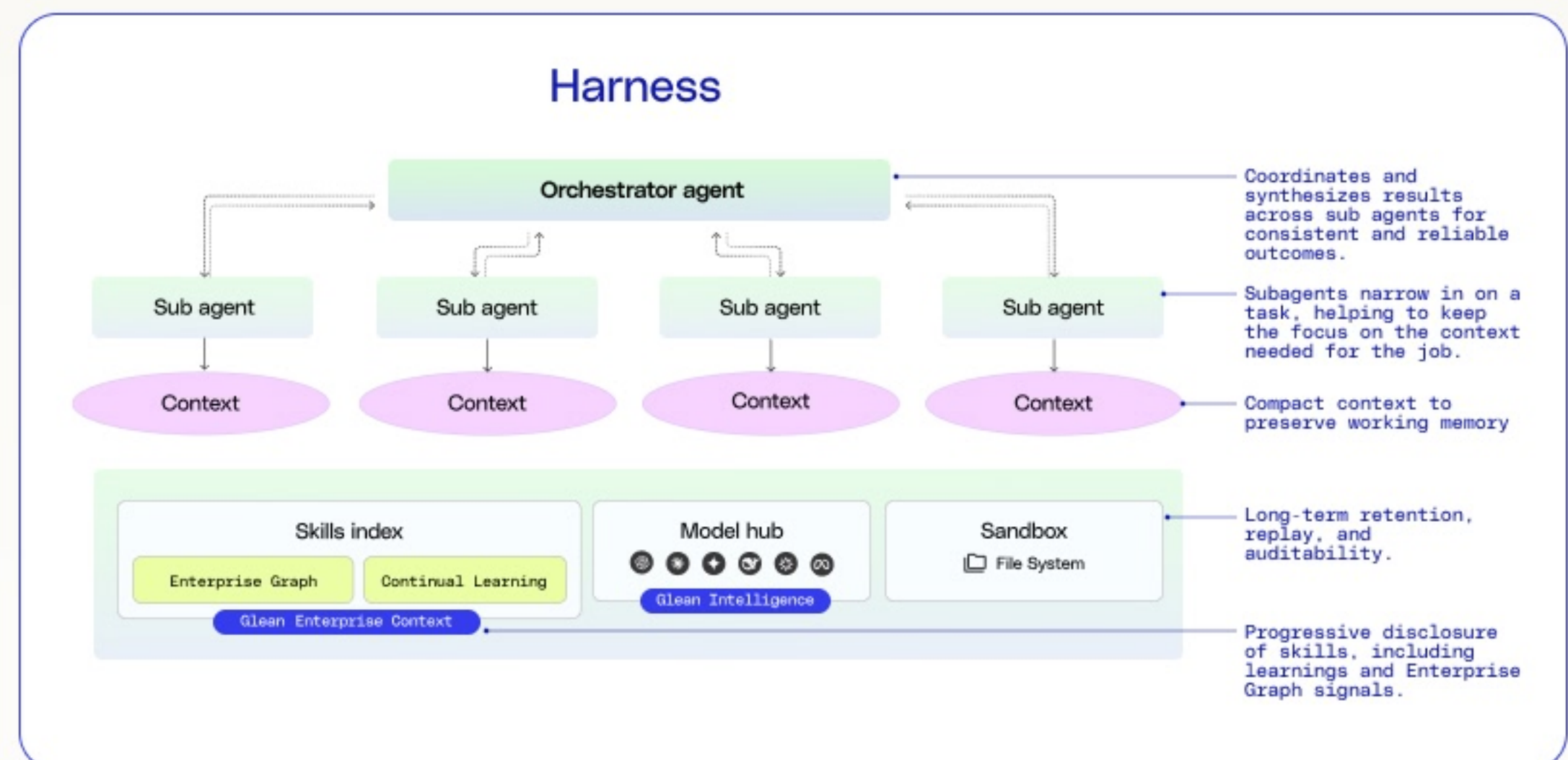
Continual learning combines process mining to improve execution with trace learning to optimize outcomes, making every agent run smarter and more efficient over time.

Harness: Orchestrating context at scale

As enterprise workflows become more agentic, the challenge becomes supporting the expanding action space available to agents while simultaneously minimizing the amount of context that must remain active for each task. In more naive architectures, taking on additional agentic work often meant continuously adding more tools, policies, and instructions directly into the system prompt, then carrying those instructions and accumulated intermediate state forward across every execution step. Over time, models were forced to repeatedly reread growing amounts of context, causing token usage to compound as tasks became longer and more complex.

The next generation of harnesses is emerging as a response to that scaling problem. In effect, these harnesses act as distributed context management systems. They continuously decide what information should remain in active reasoning, what can be compacted or externalized, and which agent or model actually requires it.

Rather than keeping an entire workflow inside a single context window, they break work across specialized agents with scoped responsibilities and isolated working memory, reducing the amount of context and action space each model must process at any given step. They progressively disclose tools so each model only reasons over the capabilities relevant to the current step. They compact intermediate outputs, move state into sandbox file systems when appropriate, and reintroduce data only when it becomes relevant again.



Harnesses act as distributed context managers, giving each agent the right context at the right step to scale long-running agentic work.

The rise of harness design reflects a broader shift in enterprise AI architecture, where the challenge is no longer simply accessing intelligence, but managing context, memory, and learning for both reliability and token efficiency.

04

The more AI understands work, the fewer tokens it needs to complete it

The same architectural foundations that make enterprise AI useful are also what make it efficient. As systems develop a deeper understanding of how work actually happens inside an organization, execution becomes less exploratory and more informed. Instead of repeatedly gathering excessive context and reasoning through every possible path, the system can operate with a stronger prior understanding of how tasks are typically completed.

That's why token efficiency is ultimately an architectural problem, not just a model problem. The systems that use tokens most efficiently are often the ones with the strongest understanding of enterprise work itself. Better context retrieval reduces unnecessary reasoning. Intelligent routing and smaller, specialized models handle execution more efficiently. Accumulated learning makes future executions more targeted. Well-designed harnesses keep context scoped and compact as tasks expand.

The point is not simply to make AI cheaper. It is to improve token yield: to generate more useful work, more reliably, from the same unit of AI spend. Enterprises that solve that problem will not just control cost better. They will build AI systems that scale more sustainably, deliver better outcomes, and compound in value over time.

Learn how Glean can help you scale your AI agents via our new investments

[Get a Demo »](#)

